极客时间

自主规划与质量控制

Planner Agent + Reviewer Agent + 自动修正循环

第 3 周 · 第 11 节

讲师：黄佳

极客时间黄佳老师出品

V2 的两个痛点

极客时间

痛点 1：固定流程

硬编码四步流水线
3 条数据 vs 200 条数据 → 同样处理
新闻快讯 vs 技术文章 → 同样 prompt

根因：没有规划环节
一刀切 → 浪费 or 炸掉

痛点 2：没有质量反馈

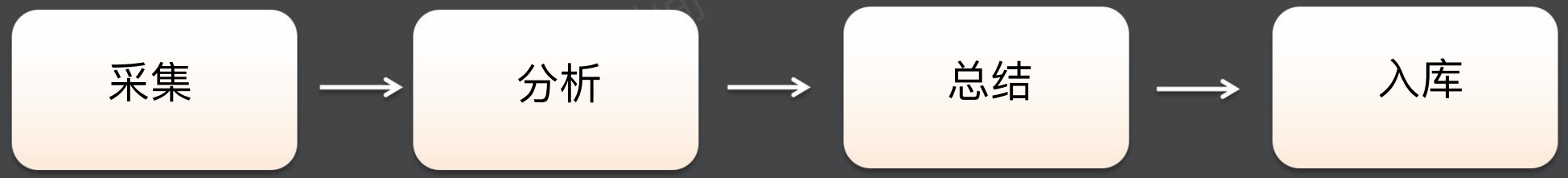
只有格式校验（JSON 合法）
摘要太短？直接入库
分析跑偏？直接入库
关键遗漏？下次还遗漏

根因：开环系统，无反馈
质量靠运气

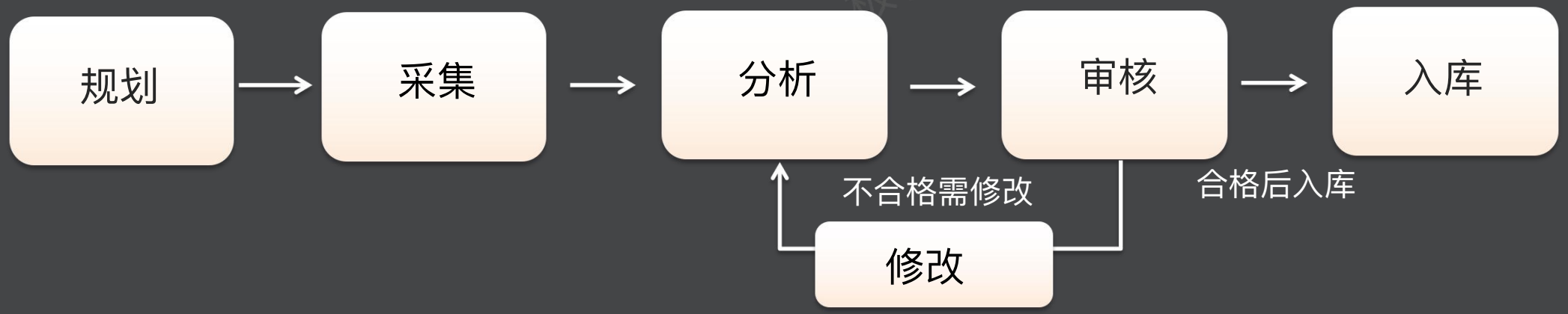
开环 vs 闭环 — V2 到 V3 的核心升级

极客时间

V2（开环系统）



V3（闭环系统）



极客时间黄佳老师出品

极客时间

Planner Agent

只规划不执行
Plan, don't execute

输入：数据概览 → 输出：结构化执行计划（JSON）

极客时间黄佳老师出品

Planner 三维度评估策略

维度	评估内容	影响决策	示例
数量	每个来源有多少条	决定批处理策略	<10 全部精读，>50 聚类取样
质量	数据完整度、信息量	决定分析深度	完整文章 → 深度分析，碎片 → 摘要
类型	新闻/技术/讨论	决定处理方式	新闻 → summarize，技术 → deep_analyze

极客时间黄佳老师出品

Planner 动态策略 — 基于数据量

- **策略 1：全量深度分析**（数据量 < 10）
- 所有条目执行 deep_analyze
- 每条预估 1500 tokens → 总成本低，不用省

- **策略 2：筛选后分析**（数据量 10–50）
- 第一步：全部做快速评分（summarize）
- 第二步：Top 10 执行 deep_analyze
- 其余保留摘要，不深入分析

- **策略 3：聚类取样**（数据量 > 50）
- 第一步：按关键词聚类分组
- 第二步：每组取最具代表性的 1–3 条 deep_analyze
- 第三步：其余标记 skip
- 大幅减少 token 消耗，同时保证覆盖度

极客时间黄佳老师出品

Planner 输出格式 — 结构化执行计划

planner output example | json

```
{
  "plan_id": "plan_20260310_001",
  "total_items": 35,
  "strategy": "filter_then_analyze",
  "tasks": [
    {
      "source": "github",
      "count": 20,
      "action": "summarize",
      "priority": "medium",
      "reason": "数量较多，先做摘要筛选"
    },
    {
      "source": "hackernews",
      "count": 10,
      "action": "deep_analyze",
      "priority": "high",
      "reason": "数量适中，逐条深度分析"
    }
  ],
  "estimated_tokens": 15000,
  "estimated_cost": 0.025
}
```

代码展示 — planner_node 实现

极客时间

workflows/planner.py | python

```python
def plan_strategy(target_count: int | None = None) -> dict:
    """根据目标采集量选择策略 —— 最小可运行 Planner"""
    if target_count is None:
        target_count = int(os.getenv("PLANNER_TARGET_COUNT", "10"))

    if target_count >= 20:
        return {"strategy": "full", "per_source_limit": 20,
                "relevance_threshold": 0.4, "max_iterations": 3,
                "rationale": f"目标 {target_count} 条，深度模式"}
    elif target_count >= 10:
        return {"strategy": "standard", "per_source_limit": 10,
                "relevance_threshold": 0.5, "max_iterations": 2,
                "rationale": f"目标 {target_count} 条，平衡模式"}
    else:
        return {"strategy": "lite", "per_source_limit": 5,
                "relevance_threshold": 0.7, "max_iterations": 1,
                "rationale": f"目标 {target_count} 条，成本优先"}


def planner_node(state: dict) -> dict:
    """LangGraph 节点：把策略写入 state[\"plan\"]"""
    plan = plan_strategy()
    print(f"[Planner] {plan[\"rationale\"]}")
    return {"plan": plan}
```

极客时间

Reviewer Agent

只评估不修改
Evaluate, don't modify

独立质检员：没有利益冲突，只看质量

极客时间黄佳老师出品

Reviewer 五维度评分体系

极客时间

维度	权重	1–3 差	4–6 中	7–10 优
摘要质量	25%	遗漏关键信息	覆盖主要信息但不精炼	准确、精炼、无冗余
技术深度	25%	只有表面描述	提到技术细节但不深入	深入分析原理和影响
相关性	20%	与主题无关	部分相关	高度相关、有实用价值
原创性	15%	纯复制粘贴	有基本改写	有独立见解或新角度
格式规范	15%	格式混乱	基本符合要求	完全符合模板

代码展示 — review_node 实现

极客时间

workflows/reviewer.py | python

```python
def review_node(state: KBState) -> dict:
    """只评估不修改：审 analyses，打 5 维度分，给结构化反馈"""
    analyses = state.get("analyses", [])
    sample = analyses[:5]  # 控 token

    prompt = REVIEWER_PROMPT.format(items=json.dumps(sample))
    result, usage = chat_json(prompt, temperature=0.1)  # 低温度保一致

    # 【关键】用代码重算加权总分——不信任模型算术
    scores = result.get("scores", {})
    weighted_total = sum(
        scores.get(d, 0) * w
        for d, w in REVIEWER_WEIGHTS.items()
    )
    passed = weighted_total >= REVIEWER_PASS_THRESHOLD

    return {
        "review_passed": passed,
        "review_feedback": result.get("feedback", ""),
        "iteration": state.get("iteration", 0) + 1,
        "cost_tracker": accumulate_usage(tracker, usage),
    }
```

```python
REVIEWER_WEIGHTS = {
    "summary_quality": 0.25,
    "technical_depth": 0.25,
    "relevance":       0.20,
    "originality":     0.15,
    "formatting":      0.15,
}
REVIEWER_PASS_THRESHOLD = 7.0
```

Reviewer 结构化反馈格式

极客时间

review_node output example | json

```
{
  "scores": {
    "summary_quality": 8,
    "technical_depth": 6,
    "relevance": 9,
    "originality": 5,
    "formatting": 8
  },
  "weighted_total": 7.15,
  "passed": true,
  "feedback": "技术深度不足，缺少底层原理分析；原创性偏低",
  "weak_dimensions": ["technical_depth", "originality"]
}
```

极客时间黄佳老师出品

带反馈修改 vs 盲目重试

极客时间

盲目重试（错误做法）

分析 → 不合格 → 再分析一遍
用同样的 prompt，同样的输入

问题：
同样的输入 ≈ 同样的输出
重试 10 次大概率还是不合格
白白浪费 token

带反馈修改（正确做法）

分析 → 审核 → 拿到反馈 → 定向修改
反馈注入修改 prompt

优势：
Reviser 知道哪里不好、怎么改
每次修改都有方向
通常 1–2 轮就能通过

极客时间黄佳老师出品

代码展示 — revise_node 实现

极客时间

workflows/reviser.py | python

```python
def revise_node(state: KBState) -> dict:
    """只修改不评估：读 feedback，LLM 定向改 analyses"""
    analyses = state.get("analyses", [])
    feedback = state.get("review_feedback", "")
    tracker = state.get("cost_tracker", {})

    if not analyses or not feedback:
        return {}

    prompt = REVISER_PROMPT.format(
        feedback=feedback,
        items=json.dumps(analyses, ensure_ascii=False),
    )

    improved, usage = chat_json(prompt, temperature=0.4)
    tracker = accumulate_usage(tracker, usage)

    if isinstance(improved, list) and improved:
        print(f"[Reviser] 定向修改 {len(improved)} 条 analyses")
        return {"analyses": improved, "cost_tracker": tracker}

    return {"cost_tracker": tracker}
```

退出策略 — 三条路径 + 上限兜底

- **max_iterations 由 Planner 决定** — 不同策略不同上限
- lite 模式 → 1 次（成本优先，不重试）
- standard 模式 → 2 次（平衡）
- full 模式 → 3 次（质量优先）
-
- **条件路由函数 route_after_review() — 三条出口：**
- 审核通过 → organize（整理入库，正常终点）
- 未通过 + 还有机会 → revise（LLM 定向修改后回到 review）
- 未通过 + 超过上限 → human_flag（标记人工介入，异常终点）
-
- **经验数据（standard 模式）：**
- 第 1 轮通过：~70% 第 2 轮通过：~25% 人工介入：~5%
-
- **为什么是 HumanFlag 而不是强制通过?**
- 超过 3 次修改还没过，说明问题不在'质量'而在'数据'——需要人判断

极客时间黄佳老师出品

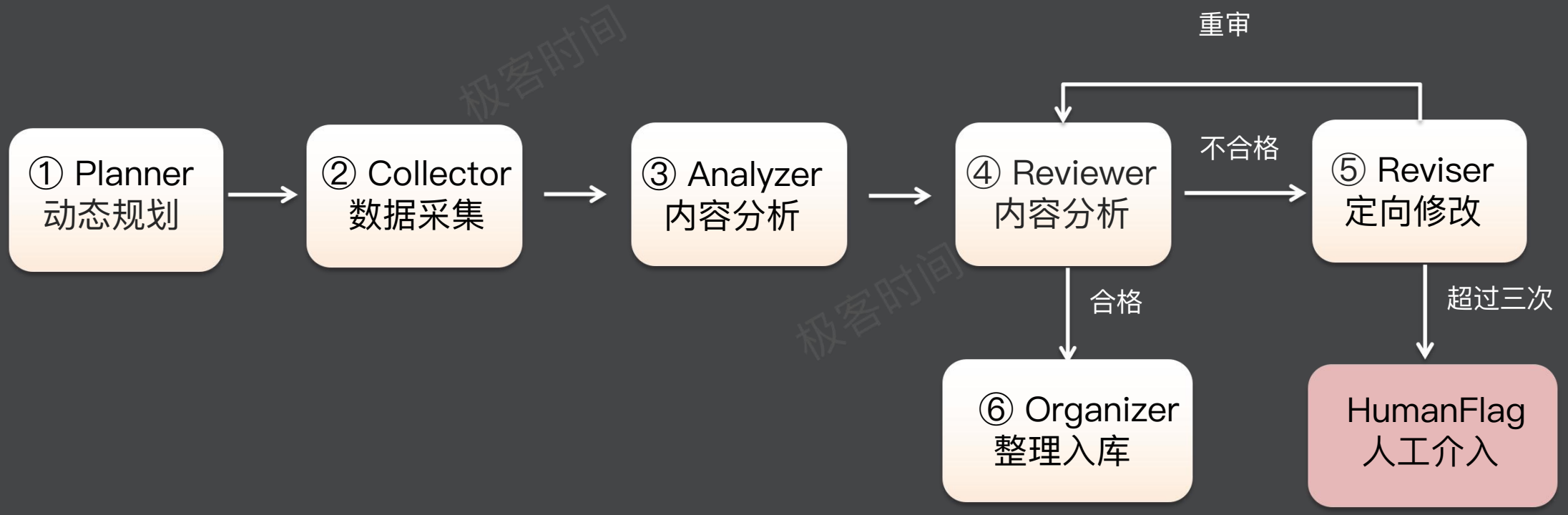
代码展示 — route_after_review 条件路由

极客时间

workflows/graph.py | python

```python
def route_after_review(state: KBState) -> str:
    """LangGraph 条件路由：审核后三条出口"""
    plan = state.get("plan", {}) or {}
    max_iter = int(plan.get("max_iterations", 3))
    iteration = state.get("iteration", 0)

    if state.get("review_passed", False):
        return "organize"      # 通过 → 整理入库（正常终点）
    elif iteration >= max_iter:
        return "human_flag"    # 超限 → 人工介入（异常终点）
    else:
        return "revise"        # 未通过 → 定向修改


# LangGraph 三路条件边
graph.add_conditional_edges(
    "review",
    route_after_review,
    {
        "organize":   "organize",   # 正常终点
        "revise":     "revise",     # 循环
        "human_flag": "human_flag", # 异常终点
    },
)

# 修改后回到审核 → 形成闭环
graph.add_edge("revise", "review")

# 两个终点都接到 END
graph.add_edge("organize", END)
graph.add_edge("human_flag", END)
```

V3 完整架构 — 6 步管线 + 审核循环



极客时间黄佳老师出品

V2 vs V3 对比

极客时间

特性	V2	V3
流程	固定 4 步	动态 6 步
规划	无	Planner 按数据量/类型规划
质量控制	格式校验	5 维度评分 + 阈值
反馈循环	无	审核→修改→重审
异常处理	直接报错	人工介入标记
适应性	一刀切	按数据量/类型调整
单次成本	~¥0.008	~¥0.015

本节作业

极客时间

任务 1：实操 1：实现 Reviewer Agent（5 维度加权评分）

在 workflows/nodes.py 实现 review_node 正式版。5 维度 prompt、REVIEWER_WEIGHTS 权重字典、代码重算加权总分（不信任模型算术）。温度设 0.1 保证评分一致性。目标文件：workflows/nodes.py

任务 2：实操 2：实现 Reviser Agent（定向修改）

在 workflows/nodes.py 实现 revise_node。读取 state['review_feedback']，把弱项维度注入修改 prompt。温度设 0.4 允许创造性改写。目标文件：workflows/nodes.py

任务 3：实操 3：端到端审核修正循环验证

运行 python -m workflows.graph，观察 6 节点流水线。故意把 REVIEWER_PASS_THRESHOLD 调到 9.0，制造未通过场景，观察 review → revise → review 循环 + human_flag 兜底。目标：验证 workflows/graph.py 的三条出口都能跑通

极客时间黄佳老师出品

本节小结

Planner 三维度评估 = 闭环的起点（复杂度 / 新鲜度 / 数据量）

Reviewer 五维度评分 = 质量守门员（>= 4.0 才通过）

带反馈修改 vs 盲目重试 —— 结构化反馈是关键

退出策略三保险 —— max 3轮 + 分数提升 + 无新建议

下一节：第 12 节 多 Agent 协作编排 — 完整 V3 工作流实现

极客时间黄佳老师出品